DESIGN AND ANALYSIS OF ALGORITHMS

Pseudocode Writing Task - Q7

PERMUTATION GENERATION

Backtracking Algorithm Design

Task	Generate all possible permutations of a set of distinct elements
Method	Backtracking with recursive calls, swapping and restoration
Input	n distinct elements stored in array A
Output	All n! permutations without repetition

This document emphasizes algorithm design: the recursive choice, swap operation, backtracking step, completeness, and complexity.

1. Problem Setup

Given n distinct elements, generate every possible arrangement of those elements. Each permutation contains all elements exactly once, but their order is different.

2. Input and Output

Item	Definition
Input array	A containing n distinct elements
Starting position	left = 0
Ending position	right = n - 1
Output	All possible permutations of A
Number of outputs	n!

3. Backtracking Idea

At each recursion level, one array position is fixed. The algorithm tries each remaining element in that position. After recursively generating the branch, the swap is undone. The restored array allows the next choice to be explored independently.

The algorithm therefore follows the pattern: choose -> recurse -> undo.

4. Final Pseudocode

```
ALGORITHM GeneratePermutations(A, left, right)

INPUT:
A = array containing n distinct elements
left = starting index
right = final index

OUTPUT:
all possible permutations of A

BEGIN
IF left = right THEN
PRINT A
RETURN
END IF

FOR i <- left TO right DO
SWAP A[left] AND A[i]

GeneratePermutations(A, left + 1, right)
```

```
SWAP A[left] AND A[i] // backtracking
END FOR
END

MAIN
READ n
READ elements into A
GeneratePermutations(A, 0, n - 1)
END
```

5. Worked Example

For A = {1, 2, 3}, the first position can contain 1, 2, or 3.

Fixed first element	Generated permutations
1	1 2 3 ; 1 3 2
2	2 1 3 ; 2 3 1
3	3 2 1 ; 3 1 2

Input

```
n = 3
Elements = 1 2 3
```

Output

```
1 2 3
1 3 2
2 1 3
2 3 1
3 2 1
3 1 2

Total permutations = 6
```

6. Swap and Backtrack

Suppose [1, 2, 3] is the current state and 2 is selected for position 0. Swapping index 0 with index 1 produces [2, 1, 3]. The recursive call works from that state. When the call returns, the same swap restores [1, 2, 3]. This restoration is the backtracking operation.

Stage	Array state	Purpose
Initial	[1, 2, 3]	Parent state
Swap	[2, 1, 3]	Fix 2 at position 0
Recurse	[2, 1, 3]	Generate remaining orderings
Backtrack	[1, 2, 3]	Restore before next choice

7. Correctness Argument

Base case: When left = right, every position is fixed. The array is a complete permutation and is printed.

Recursive case: The loop tries every remaining element at the current position and recursively arranges all positions after it.

Restoration: The second swap returns the array to its state before the branch, so later branches are not affected.

Completeness: Every possible choice for every position is considered, so every permutation is generated.

No repetition: With distinct elements, each sequence of choices gives one unique permutation.

8. Complexity

Measure	Result
Total permutations	n!
Generation time	O(n * n!)
Auxiliary recursion space	O(n)
Array handling	In-place swaps; no full copy required per branch

The exponential growth is unavoidable when the requirement is to explicitly output all n! permutations.

9. Requirement Check

Requirement	Covered
Input defined	A, n, left, right
Output defined	All permutations printed at base case
Loops	FOR i from left to right
Recursive calls	GeneratePermutations(A, left + 1, right)
Swapping	Swap current position with candidate position
Backtracking	Undo the swap after recursion
No repetition	Distinct elements are selected once per position

10. Submission Structure

```
CO4_AT1/
├── Question_1/
│   ├── Source_Code/
│   │   └── permutation_pseudocode.txt
│   └── Report/
│   └── Q7_Permutation_Generation_Pseudocode_Report.pdf
└── README.md
```

11. Final Answer

The complete backtracking solution fixes one position, tries every remaining element using a swap, recursively generates the remaining arrangements, and swaps back after the recursive call. For n distinct elements, this produces exactly n! permutations without repetition.